COAL ASSIGNMENT #02

Name : Muhammad Hamdan
Roll number : 25K-0136
Section : BAI-3A

Q1

```
INCLUDE Irvine32.inc

.code
main PROC
    mov ax, 3          ; AX = 3
    mov bx, ax
    shl bx, 1          ; BX = 3x2 = 6
    mov cx, ax
    shl cx, 3          ; CX = 3x8 = 24
    shl ax, 4          ; AX = 3x16 = 48
    add ax, bx         ; AX = 48 + 6 = 48
    add ax, cx         ; AX = 48 + 24 = 78
                       ; 26 * 3 = 78
    exit
main ENDP
END main
```

Q2

```
INCLUDE Irvine32.inc

.data
    num BYTE 0CDh, 0ABh, 45h, 23h

.code
main PROC
```

```
        mov   cx, 4
R1:
        shr   num+3, 1
        rcr   num+2, 1
        rcr   num+1, 1
        rcr   num+0, 1
        Loop  R1:
                                ; num = 02 34 5A BCh
$       mov   num+0, 0CDh
        mov   num+1, 0ABh
        mov   num+2, 45h
        mov   num+3, 23h
          mov   cx, 4
L1:
        shl   num+0, 1
        rcl   num+1, 1
        rcl   num+2, 1
        rcl   num+3, 1
        loop  L1

        exit
main    ENDP
END     main
```

Q3

```
INCLUDE Irvine32.inc
.data
    num WORD 0ABCDh, 2345h

.code
main PROC
    mov ecx, 2
R1:
    shr num+2, 1
    rcr num+0, 1
    loop R1
    mov num+0, 0ABCDh
    mov num+2, 2345h
    mov ecx, 2
L1:
    shl num+0, 1
    rcl num+2, 1
    loop L1

    exit
main ENDP
END main
```

.data

Bsec BYTE ?

Q4 Bmin BYTE ?

mov dx, 0101100111011010b

(a) mov al, dl
and al, 00011111b
mov Bsec, al ; al = 00110101b
; 26 sec

(b) mov ax, dx
shr ax, 5
and al, 00111111b ; al = 00001110b
mov Bmin, al ; 14 min

Q5

DX = 9A40h

AX = 918Ah

CF = 1

dx = 9A3Fh
ax = 7FB0h
+ 90EAh CF=0
106A

dx = 9A3F CF=1
+1
+1 (CF)
dx = 9A41 here CF=0

stc CF=1
AX = 106A+
+8120
918A CF=0

dx = 9A41
FFFF
+0 = CF
dx = 19A40 CF=1

Q6

(a) test ax, ax ; 0 & 0 = 0

(b) test bx, 1 ; LSB of odd num must be 1

(c) test dx, dx ; SF = top bit of Dx

(d) test dx, 8000h ; it check only sign bit

(e) test bl, 0FFh ; bl & FF = bl, PF = 1 only when bl have even count of 1

Q7

~~DX = 0001 0000b~~

dx ~~= 16h~~ 5d = 5h

ax = 10h

dx = 5 + 10 = 15h

dx = 5 + 10 + 1 = 16h

dx = 5 = 0101 CF = 0

stc CF = 1

0101 a=10=1010

+ 1 of CF

0110

+ CF = 1

CF = 0

dx 0101

ax 1010

+1 of CF

1 0000 CF = 1

Final answer :

DX = 16h = 22d

Q8

val1

Errors:

1. mov esi, val1 loads the value, not the address, use OFFSET val1 & OFFSET val2.
2. dec esi / dec edi go backwards. Use inc (low to high)
3. The result overwrites val1. Store it in result instead
4. mov cx, 8 set only cx, but loop uses ECX. Use mov ecx, 8.

corrected code:

```
.code
    main PROC
    more  ecx, 8
    mov  esi, OFFSET val1
    mov  edi, OFFSET val2
    mov  ebx, OFFSET result
    clc
top:
    mov al, BYTE PTR [esi]
    sbb al, BYTE PTR [edi]
    mov BYTE PTR [ebx], al
    inc esi
    inc edi
    in  ebx
    Loop top
.end main
```

Q9

```
INCLUDE   Irvine32.inc

.data
    msg  BYTE  "Enter marks (0-100): ", 0

.code main PROC
    mov  edx, OFFSET msg
    call WriteString
    call ReadInt
```

```
cmp eax, 80
jge grade A
cmp eax, 60
jge grade B
cmp eax, 40
jge grade C

mov al, 'F'
jmp show

grade A:
    mov al, 'A'
    jmp show
grade B:
    mov al, 'B'
    jmp show
grade C:
    mov al, 'C'
    jmp, show

show:
    call writeChar
    call Crlf
    exit
main ENDP
END main
```

R.W

Q10

EAX = 3
EBX = 10
Stack = empty

EAX	EBX	Stack
3	?	empty
3	7	empty
3	7	3
3	7	3, 7
7	7	3
7	7	3, 10
7	10	3, 7
3	10	empty

Q11 INCLUDE Irvine32.inc

```
.data
    num1    SDWORD ?
    num2    SDWORD ?
    num3    SDWORD ?
    num4    SDWORD ?
    result  SDWORD ?

.code
main PROC
    mov  eax, num1
    cmp  eax, num2
    jle  elseif

    mov  eax, num3
    cmp  eax, num4
    je   elseif

    mov  eax, num1
    add  eax, num3
    mov  result, eax
    jmp  done
```

```
elseif:
    mov eax, num2
    cmp eax, num4
    jne elseBlock
    mov eax, num2
    sub eax, num1
    mov result, eax
    jmp done

elseBlock:
    mov result, 0

done:
    exit
main ENDP
END main
```

Q12

```
INCLUDE Irvine32.inc

.data
    op1 SDWORD ?
    op2 SDWORD ?
    x   SDWORD ?
    y   SDWORD ?
    z   SDWORD ?

.code
main PROC
```

```
whileloop:
        mov   eax, op1
        cmp   eax, op2
        jg    endWhile
      mov  eax, op1
      cmp  eax, x
      jl   elsePart
      cmp  eax, y
      jg   elsePart
           add   z, 10
           jmp   next

elsePart:
        sub   z, 10

next:
        dec   op1
        jmp   whileloop

endWhile:
        exit
        main  ENDP
END     main.
```